LABORATORY MANUAL · WEEK 3

Lab 03 — Git in Depth: Branching, Rebasing and Finding Regressions

DevOps Principles and Practices (2+1) · BS Computer Science · National Textile University

Duration	3 hours (one laboratory session)
Mapped outcome	L-CLO-1 — Construct an automated continuous integration pipeline using industry-standard tools. (P3 — Precision, Psychomotor)
Graduate attribute	GA-5 — Modern Tool Usage
Prerequisite	Lab 02 complete — a working `Makefile` with `check`, `build`, `test`, `clean` and `ci` targets
Environment	Ubuntu 24.04 under WSL2 (or native Linux / macOS Terminal). Git 2.34 or newer.
Deliverable	A .zip of your repository plus a short report — see Section 9
Weight	Counts towards the 35 % weekly laboratory task component

1. Why this lab follows Lab 02

You have a `Makefile` that builds only what changed. It exists in exactly one place: a folder on your laptop. If that laptop fails tonight, the work is gone, and there is no record of how it came to look the way it does.

This week's lecture explained what Git actually stores — blobs, trees, commits, and refs that point at them. This laboratory is where you handle those objects yourself, on your own project, and then use them to do something you cannot do any other way: find the exact commit that broke a build, out of sixteen candidates, in four tests.

Three things make this lab different from the Git you have been using:

1. You will **look inside** the repository rather than only issuing commands at it. `git cat-file` will show you the objects from the lecture, made from your own files.
2. You will **cause a merge conflict on purpose**. Every conflict you have met so far arrived under deadline pressure. One produced deliberately, in a calm room, is worth ten survived in panic.
3. You will **rewrite history** — and learn precisely where the line is that you must not cross.

> **The rule that governs everything today.** Git never edits anything. An object's name is the hash of its contents, so changing contents means creating a new object. Every command in this lab either adds objects or moves a pointer. If you keep that sentence in mind, nothing today is surprising.

2. Objectives

By the end of this session you will be able to:

- Place an existing project under version control and decide what must **not** be tracked.
- Explain the difference between the working directory, the index and `HEAD`, and prove it with `git cat-file`.
- Write commits that a stranger can read, and amend one that is not yet shared.
- Create a branch, merge it with `--no-ff`, and resolve a conflict deliberately rather than by guessing.
- Rebase a branch, squash several commits into one with an interactive rebase, and say why the hashes changed.
- Tag a release, and locate a regression with `git bisect` in logarithmic rather than linear time.

3. Before you start

3.1 Tell Git who you are

Git records an author on every commit. If you have never set this, do it now — commits made without it are awkward to fix later.

```
git config --global user.name  "Your Name"
```

```
git config --global user.email "your.email@example.com"
git config --global init.defaultBranch main
git config --global core.editor nano
```

The third line makes `git init` create a branch called `main` rather than `master`. The fourth decides which editor opens when Git needs one — you will need it in Task 4. If you prefer `vim` and know how to leave it, use that instead.

> **nano is the safe choice.** In Task 4 Git will open an editor and wait. If that editor is `vim` and you have not used it, you will be stuck. In `nano`, you edit normally, then press Ctrl+O and Enter to save, and Ctrl+X to exit. The manual assumes `nano`.

3.2 Get your Lab 02 work into place

Everything today is done on your own Lab 02 project. Put it somewhere clean:

```
mkdir -p ~/devops/lab03
cd ~/devops/lab03
cp -r ~/devops/lab02/Makefile ~/devops/lab02/src  .
ls
```

If you no longer have your Lab 02 work, the provided `fallback/` folder contains a working `Makefile` and the three `src/*.txt` inputs. Use it only if you must — and say so in your report.

Confirm the pipeline still runs before you place it under version control. If this fails, fix it now; nothing later in the lab will work.

```
make ci
```

Expected output

```
rm -rf out
check: 3 source file(s) in src
mkdir -p out
  BUILD src/greeting.txt -> out/greeting.txt
  BUILD src/notes.txt -> out/notes.txt
  BUILD src/services.txt -> out/services.txt
TEST PASSED
```

3.3 Check your Git version

```
git --version
```

Anything from 2.34 onwards is fine. The lab uses `git switch` and `git restore`, which arrived in 2.23; if your Git is older than that, tell the instructor rather than substituting `git checkout` on your own.

Task 1 Put the pipeline under version control — and look at what Git stored — 25 minutes

Not assessed directly, but everything else depends on it.

4.1 Create the repository

```
git init -b main .
```

Git has created one hidden folder, `.git`, and changed nothing else. Look at what is now in it, then ask Git what it sees:

```
ls -a
git status --short
```

Expected output

```
?? Makefile
?? out/
?? src/
```

Three untracked entries, and one of them is wrong. `out/` holds files your `Makefile` generates. They are not source; they can be reproduced at any moment by running `make`; and committing them would mean every build produces a spurious change for your teammates to merge.

> **A rule you will apply for the rest of your career.** Never commit generated files. Source goes in version control; build outputs come out of a build. Week 5 gives the outputs a proper home — an artifact repository — and Week 9 rebuilds them inside a container. For now, exclude them.

4.2 Tell Git what to ignore

Create a file called `.gitignore` containing exactly two lines:

```
out/
*.log
```

Then look again:

```
git status --short
```

Expected output

```
?? .gitignore
?? Makefile
?? src/
```

`out/` has disappeared from the listing and `.gitignore` itself has appeared. Note that `.gitignore` **is** committed — the decision about what to ignore is part of the project, and every teammate needs it.

4.3 Make the first commit

Stage everything, then commit with a message that has a subject line, a blank line, and a body:

```
git add .
git commit
```

Your editor opens. Type this, then save and exit:

```
Add Lab 02 build pipeline

Makefile builds src/*.txt into out/ and verifies the result.
out/ is generated, so it is ignored.
```

```
git log --oneline
```

Expected output

```
6b1fe51 Add Lab 02 build pipeline
```

> **Your hashes will not match the manual.** A commit's hash is computed from its contents — which include the author name, the email and the timestamp. Yours differ from mine, so every hash printed from here on will differ from yours. That is correct behaviour, not an error. The one exception is Task 5, where the hashes are fixed on purpose.

4.4 Now look at what Git actually stored

This is the part of the lecture you could not do from a slide. Ask Git to print the commit object it just wrote:

```
git cat-file -p HEAD
```

Expected output

```
tree c956df882a25edc172859c82587b912fd67c9503
author Student Name <student@ntu.edu.pk> 1791027975 +0500
committer Student Name <student@ntu.edu.pk> 1791027975 +0500

Add Lab 02 build pipeline

Makefile builds src/*.txt into out/ and verifies the result.
out/ is generated, so it is ignored.
```

That is the **entire** commit. A tree, an author, a committer, a message — and no parent, because this is the first commit in the repository. Now follow the tree pointer:

```
git cat-file -p HEAD^{tree}
```

Expected output

```
100644 blob 23f53531689a88d8f627f7693cdc57de2c6ec079   .gitignore
100644 blob 558b4492bf4108cf6310a54d65956591d7ae3183   Makefile
040000 tree dd92d4dbdf1c125482c3e27190da6e727ca5b168   src
```

Two blobs and a sub-tree, exactly as the lecture described. `src` is a tree because it is a directory. Follow it one level deeper, and then read a blob by its hash:

```
git cat-file -p HEAD^{tree}:src
git cat-file -p HEAD:src/services.txt
```

Expected output

```
payment-service
auth-service
cart-service
```

You have just walked the object graph from a branch name to the contents of a file, by hand, in four commands. **Record the three hashes above in your report** — your own, not mine.

4.5 Checkpoint

- `git status --short` prints nothing (a clean tree).
- `out/` is absent from `git status` but still present on disk.
- You can explain what a blob, a tree and a commit are, using output from your own repository.

Task 2 The staging area, and history worth reading — 25 minutes

Not assessed directly. Task 4 is assessed and depends on this.

5.1 Prove the three trees are three different things

The lecture claimed that the working directory, the index and `HEAD` can hold three different versions of the same file at the same time. Most students do not believe this until they see it. Do exactly this, in order:

```
printf 'first version\n'  > src/report.txt
git add src/report.txt
printf 'SECOND version\n' > src/report.txt
```

You created a file, staged it, then edited it again without staging. Now ask all three where they stand:

```
git status --short
cat src/report.txt
git cat-file -p :src/report.txt
```

Expected output

```
AM src/report.txt
SECOND version
first version
```

Read the three lines carefully. `AM` means **A**dded to the index and **M**odified since. The working directory holds `SECOND version`. The index — the thing a commit is actually built from — still holds `first version`. The `:` prefix in `git cat-file -p :src/report.txt` means "read this path from the index".

5.2 Commit, and watch the wrong version land

```
git commit -m "Add report.txt to the build inputs"
git cat-file -p HEAD:src/report.txt
cat src/report.txt
git status --short
```

Expected output

```
first version
SECOND version
 M src/report.txt
```

> **This is the single most common Git confusion.** You committed `first version`, because that is what was in the index. Your edit is still sitting in the working directory, uncommitted. Every "I committed but my change is missing" complaint you will ever hear is this, and now you have caused it deliberately and watched it happen.

Fix it the ordinary way — stage the current contents and commit again:

```
git add src/report.txt
git commit -m "Use the corrected report.txt"
```

5.3 Amend: rewriting a commit you have not shared

That last message is poor. It describes the mechanics, not the reason. Because nothing has been pushed anywhere, you may rewrite it:

```
git rev-parse --short HEAD
git commit --amend -m "Correct the contents of report.txt"
git rev-parse --short HEAD
git log --oneline
```

Expected output

```
2e5842d
3f91ca0
3f91ca0 Correct the contents of report.txt
```

```
3813b68 Add report.txt to the build inputs
6b1fe51 Add Lab 02 build pipeline
```

The hash changed. `--amend` did not edit the old commit — it could not, because an object's name is the hash of its contents. It built a **new** commit with the corrected message and moved the branch to point at it. The old one is still in the object database, orphaned, until Git garbage-collects it.

> **The line you must not cross.** Amending is safe here only because this commit exists nowhere but your laptop. The moment a commit has been pushed, or pulled by anyone else, rewriting it makes your history disagree with theirs. Task 4 returns to this.

5.4 Commit messages a stranger can read

You are writing for an engineer debugging at two in the morning, six months from now, who has no other source of information about why this change exists. That engineer is usually you.

Instead of	Write
`fix`	`Reject enrolment after the fee deadline`
`updated makefile`	`Add a count target so builds report their size`
`stuff`	`Ignore out/ so generated files stay out of history`

- Subject line in the imperative mood, under about 50 characters, no full stop.
- Then a blank line. Git treats the first line specially; without the blank line, tools cannot separate subject from body.
- Then the body: **why**, not what. The diff already shows what changed.

5.5 Checkpoint

- You can state what the index is, and why a commit can differ from your files on disk.
- `git log --oneline` shows three commits with readable subjects.
- You can say why `--amend` produced a different hash.

Task 3 Branch, merge, and a conflict you cause on purpose — 40 minutes

Not assessed directly. The judgement it builds is assessed in Task 4 and in the viva.

6.1 Branch, and add a feature

```
git switch -c feature/word-count
```

`switch -c` creates the branch and moves onto it. Confirm with `git status`. Now add a target to the end of your `Makefile` that reports how many words were built:

```
count: build  ## report how many words were built
	@wc -w $(OUTPUTS) | tail -1
```

> **That indent is a TAB.** As in Lab 02, every recipe line in a Makefile must begin with a real tab character. If you copy this out of the PDF you will get spaces and make will say `missing separator`. Check with `cat -A Makefile` — a real tab shows as `^I`.

```
make count
git commit -am "Add a count target reporting built words"
```

6.2 Meanwhile, main moves

A teammate had the same idea, and solved it differently. Simulate that by going back to `main` and adding a **different** `count` target in the same place:

```
git switch main
```

Append this to the end of your `Makefile`:

```
count: build  ## count the source files
	@ls -1 $(SRC_DIR)/*.txt | wc -l
```

```
git commit -am "Add a count target for source files"
```

Two branches have now changed the same region of the same file in different ways. This is the exact situation the lecture drew. Merging is about to fail, and that is the point.

6.3 Cause the conflict

```
git merge --no-ff feature/word-count -m "Merge feature/word-count"
```

Expected output

```
Auto-merging Makefile
CONFLICT (content): Merge conflict in Makefile
Automatic merge failed; fix conflicts and then commit the result.
```

```
git status --short
```

Expected output

```
UU Makefile
```

`UU` means both sides modified the file and neither won. Open the `Makefile` and find the region Git has marked:

Expected output

```
<<<<<<< HEAD
count: build  ## count the source files
	@ls -1 $(SRC_DIR)/*.txt | wc -l
=======
count: build  ## report how many words were built
	@wc -w $(OUTPUTS) | tail -1
```

```
>>>>>>> feature/word-count
```

- Everything between `<<<<<<< HEAD` and `=======` is **ours** — the branch you are standing on, `main`.
- Everything between `=======` and `>>>>>>>` is **theirs** — the branch being merged in.
- Git is not reporting a failure. It is refusing to guess, which is the only correct thing it can do.

6.4 Resolve it deliberately

Resolving is **not** picking a side. Both behaviours are useful, so the correct answer here is neither version as written. Replace the whole marked region — markers included — with one target that does both:

```
count: build  ## report source files and built words
	@ls -1 $(SRC_DIR)/*.txt | wc -l
	@wc -w $(OUTPUTS) | tail -1
```

Make sure no <, = or > marker lines remain anywhere in the file, then mark it resolved and complete the merge:

```
grep -n '<<<<<<<\|=======\|>>>>>>>' Makefile   # must print nothing
make count
git add Makefile
git commit --no-edit
```

Expected output

```
  BUILD src/report.txt -> out/report.txt
       4
      31 total
```

The first line appears because `count` depends on `build`, and `report.txt` had never been built. The `4` is your source file count; the `31` is the word count of the built outputs. Your numbers may differ slightly.

6.5 Look at what a merge commit is

```
git cat-file -p HEAD | head -3
```

Expected output

```
tree 575350e7a0867d0eec0c9caf59055236c0f27a6f
parent 341963697c3de78c8154e4293c5a0ff23a001fb2
parent cd36a1f9e676f2bd6662cc76cc798ac2cf57fd20
```

Two parents. That is the entire definition of a merge commit, and it is why the merge is visible in the history forever. `--no-ff` forced this commit to exist; without it, where a fast-forward is possible, Git would simply have moved the branch pointer and left no record that a branch had ever existed.

```
git log --oneline --graph | head -6
```

6.6 Checkpoint

- `git log --graph` shows the two lines of development rejoining.
- `make count` runs and prints both numbers.
- You can explain why resolving a conflict is not the same as choosing a side.

Task 4 ASSESSED — rebase, squash, and tag a release 45 minutes

> **This task is marked.** Section 10 is the rubric. The commands matter less than being able to say what each one did to the object graph — the viva will ask.

7.1 Make a mess worth cleaning up

Real feature branches accumulate commits nobody should ever read. Create three of them deliberately, on a new branch, each appending one comment line to the end of your `Makefile`:

```
git switch -c feature/docs

echo "# count: see section 5 of the README"   >> Makefile && git commit -am "wip"
echo "# OUTPUTS is set by the wildcard above" >> Makefile && git commit -am "fix"
echo "# run 'make help' for all targets"      >> Makefile && git commit -am "more"

git log --oneline -3
```

Expected output

```
064d217 more
849b4bd fix
8447ecb wip
```

Three commits, three useless messages. No reviewer can work with this, and `git bisect` in Task 5 would be hampered by it. Collapse them into one.

7.2 Interactive rebase

```
git rebase -i HEAD~3
```

`HEAD~3` means "the last three commits". Your editor opens with a plan — **oldest commit first**, which is the opposite order to `git log`:

Expected output

```
pick 8447ecb wip
pick 849b4bd fix
pick 064d217 more

# Rebase 3f91ca0..064d217 onto 3f91ca0 (3 commands)
#
# Commands:
# p, pick <commit> = use commit
# s, squash <commit> = use commit, but meld into previous commit
# ...
```

Change the word `pick` to `squash` on the **second and third** lines only. The first must stay `pick` — it is the commit the others will be folded into. Leave everything else alone:

Expected output

```
pick 8447ecb wip
squash 849b4bd fix
squash 064d217 more
```

Save and exit (in `nano`: Ctrl+O, Enter, Ctrl+X). Git immediately opens a second editor, this time for the combined commit message. Delete everything there and write one real message:

```
Document the Makefile targets inline
```

Save and exit again.

```
git log --oneline -2
```

Expected output

```
e1acc4b Document the Makefile targets inline
dcb460e Merge feature/word-count
```

Three commits became one. The three originals still exist as orphaned objects, but nothing points at them any more.

> **If the rebase goes wrong, you are not stuck.** `git rebase --abort` returns you to exactly where you started, at any point during a rebase. Use it without embarrassment; it is what it is for.

7.3 Rebase onto a main that has moved

While you were working, `main` gained a commit. Simulate it, then bring your branch up to date:

```
git rev-parse --short HEAD        # note this value

git switch main
printf '# Build pipeline\n\nSee the Makefile.\n' > README.md
git add README.md
git commit -m "Add a README"

git switch feature/docs
git rebase main
git rev-parse --short HEAD        # and this one
```

Expected output

```
e1acc4b
Rebasing (1/1)
Successfully rebased and updated refs/heads/feature/docs.
b665d45
```

The hash changed, and you changed nothing. Your commit now has a different parent — the new tip of `main` — and the parent is part of the hashed content, so it is a different object with a different name. This is exactly the mechanism behind the golden rule.

> **The golden rule of rebasing.** Never rebase commits that exist outside your own machine. Anyone who already had the old hash now holds a commit that no longer exists on the remote, and their history and yours have silently forked. Rebase your own branch freely; after you share, add commits instead of replacing them.

7.4 Merge and tag the release

```
git switch main
git merge --no-ff feature/docs -m "Merge feature/docs"

git tag -a v1.0.0 -m "First complete build pipeline"
git tag -n
git describe --tags
git cat-file -t v1.0.0
```

Expected output

```
v1.0.0          First complete build pipeline
v1.0.0
tag
```

`-a` creates an **annotated** tag: a real object in the database with its own message and author, which is what the last command proves. A plain `git tag v1.0.0` would create only a lightweight ref. Releases should always be annotated — Week 05 relies on this when it identifies build artifacts by version.

```
git log --oneline --graph | head -8
make ci
```

Confirm the pipeline still passes on the merged result. A tagged release that does not build is worse than no tag at all.

7.5 Checkpoint — this is what is marked

- `git log --oneline --graph` shows two merge commits and no `wip`, `fix` or `more`.
- `git tag -n` shows `v1.0.0` with a message, and `git cat-file -t v1.0.0` prints `tag`.
- `make ci` passes at the tagged commit.
- You can explain, in one sentence each, why `--amend` and `git rebase` both produce new hashes.

Task 5 Find a regression with git bisect — 25 minutes

Not assessed directly, but it is the payoff for everything in Task 2 — and the most useful single command in this manual.

8.1 The situation

A pipeline that worked a fortnight ago is now producing wrong output. Sixteen commits have landed since. Reading sixteen diffs would take an hour; `git bisect` will find the culprit in four tests, because it performs a binary search over history.

Build the repository. The generator is provided — typing sixteen commits by hand would teach you nothing:

```
cd ~/devops/lab03
bash make-bisect-repo.sh
cd regression-demo
```

Expected output

```
Created regression-demo with 16 commits on main.
Somewhere in there, transform.sh started dropping a line. Find it.
```

8.2 See the failure for yourself

```
./transform.sh
wc -l < input.txt
wc -l < output.txt
cat output.txt
```

Expected output

```
       4
       3
PAYMENT-SERVICE
AUTH-SERVICE
CART-SERVICE
```

Four lines went in; three came out. `search-service` has vanished, and the script exited successfully while doing it. This is the kind of failure that reaches production: nothing crashed, nothing was logged, the answer was simply wrong.

8.3 Write a test that decides good from bad

`git bisect` can drive itself if you give it a script that exits `0` when the code is good and non-zero when it is bad. Create `test-build.sh` in the `regression-demo` folder:

```
#!/usr/bin/env bash
# test-build.sh — exit 0 if the transform is correct, 1 if it is broken.
./transform.sh
want=$(wc -l < input.txt)
got=$(wc -l < output.txt)
test "$want" = "$got"
```

```
chmod +x test-build.sh
./test-build.sh ; echo "exit code: $?"
```

That must print `exit code: 1` on the current, broken code. A test that cannot fail is worthless, so always check it fails before you trust it.

> **Why the last line has no `if`.** The exit status of a script is the exit status of its last command, and `test` already exits 0 or 1. Writing `if ... then exit 0 else exit 1 fi` would work too, but this is the idiom you will meet in real pipelines.

8.4 Run the bisect

Bisect needs one commit known to be bad and one known to be good. `HEAD` is bad. The very first commit is good:

```
git bisect start HEAD $(git rev-list --max-parents=0 HEAD)
git bisect run ./test-build.sh
```

Expected output

```
Bisecting: 7 revisions left to test after this (roughly 3 steps)
running './test-build.sh'
Bisecting: 3 revisions left to test after this (roughly 2 steps)
running './test-build.sh'
Bisecting: 1 revision left to test after this (roughly 1 step)
running './test-build.sh'
Bisecting: 0 revisions left to test after this (roughly 0 steps)
running './test-build.sh'
2d389cb76ad43436966cfd3078939815c401214d is the first bad commit
commit 2d389cb76ad43436966cfd3078939815c401214d
Author: Pipeline Team <team@novasoft.example>
Date:   Mon Feb 9 09:00:00 2026 +0500

    Speed up the transform step

 README.md    | 1 +
 transform.sh | 3 ++-
 2 files changed, 3 insertions(+), 1 deletion(-)
bisect found first bad commit
```

> **These hashes WILL match yours.** The generator pins the author, the email and every commit date, so the repository is byte-identical on every machine. If your first bad commit is not `2d389cb`, something is wrong — most likely you edited a file before bisecting. Delete the folder and regenerate it.

Four test runs to search sixteen commits. Now look at what that commit actually did, then put the repository back:

```
git show 2d389cb -- transform.sh
git bisect reset
```

Someone replaced a correct `tr` pipeline with one that first pipes the input through a `sed` expression deleting the last line — and described it in the commit message as a speed-up. The tests that existed did not check the line count, so nothing caught it.

8.5 Why Task 2 mattered

Bisect works by checking out arbitrary historical commits and testing them. That only works if **every commit builds and does one thing**. A history full of `wip` commits that do not run, or commits that change five unrelated things at once, degrades bisect from a four-step search to a guess. Sloppy history is not untidy — it disables the best debugging tool you have.

9. Submission

Submit **one** `.zip` named `Lab03_<your-roll-number>.zip` containing:

1. Your `lab03` project folder **including the `.git` directory** — that is where the marks are. Do not run `git clean` or delete it.
2. A report, `report.md` or `report.pdf`, of roughly one page.

9.1 What the report must contain

- The three object hashes you recorded in Section 4.4, with one line saying what each object is.
- Your `git log --oneline --graph` output for `main`, pasted as text.
- One short paragraph: why did the hash change in Section 7.3 when you changed no files?
- One short paragraph: your conflict in Task 3 — what each side wanted, and why your resolution is neither of them.
- The first bad commit from Task 5, and one sentence on why `git bisect` needed only four tests for sixteen commits.
- If you used the provided `fallback/` Makefile rather than your own Lab 02 work, say so.

> **Zip the folder, not the contents.** From `~/devops`, run `zip -r Lab03_<roll>.zip lab03`. Zipping from inside the folder often omits the hidden `.git` directory, and without it there is nothing to mark.

```
cd ~/devops
zip -r Lab03_21-NTU-CS-1234.zip lab03
unzip -l Lab03_21-NTU-CS-1234.zip | grep -c '\.git/'
```

That last command must print a number greater than zero. If it prints `0`, your `.git` directory is missing and the submission cannot be marked.

10. Marking rubric

Task 4 carries the marks; the other tasks are assessed indirectly through the report and the viva.

Criterion	Weight	What full marks looks like
Repository hygiene	15 %	`out/` ignored and absent from history; `.gitignore` committed; no generated files tracked.
Commit quality	20 %	Imperative subjects under 50 characters, bodies explaining why. No `wip`, `fix` or `more` surviving in `main`.
Branching and merge	20 %	Feature branch merged with `--no-ff`; conflict resolved to a working target that is neither side verbatim.
Rebase and squash	25 %	Three commits squashed into one; branch rebased onto a moved `main`; student can explain why the hashes changed.
Tagging	10 %	Annotated `v1.0.0` (`git cat-file -t` prints `tag`), and `make ci` passes at that commit.
Report	10 %	All six points in Section 9.1 answered, in the student's own words.

> **Two things that cost marks every year.** Committing `out/` — it shows up immediately in `git log --stat` and costs the whole 15 %. And squashing on the wrong line in Task 7.2: leaving `squash` on the first line makes the rebase fail, and the usual reaction is to force something through rather than run `git rebase --abort` and start again.

11. Troubleshooting

Symptom	Cause and fix
`Makefile:NN: *** missing separator. Stop.`	A recipe line begins with spaces, not a tab — almost always from pasting out of the PDF. Check with `cat -A Makefile`; a real tab shows as `^I`.
`Please tell me who you are`	You skipped Section 3.1. Set `user.name` and `user.email`, then retry the commit.
An editor opened and you cannot leave	If it is `vim`: press `Esc`, then type `:wq` and Enter. Then set `git config --global core.editor nano` so it does not happen again.
`error: Committing is not possible because you have unmerged files`	A conflict is still unresolved. Run `git status`, edit every file listed as UU, remove all marker lines, then `git add` each one.
Conflict markers left in the file	The build fails in confusing ways. `grep -n '<<<<<<<' Makefile` finds them. All three marker lines must go, not just the first.
Rebase stopped half way and you are lost	`git rebase --abort` returns you to exactly where you began. Nothing is lost.
`detached HEAD` **during a bisect**	Normal — bisect checks out commits directly. `git bisect reset` returns you to your branch when you are done.
Task 5 finds a different commit	You edited a file in `regression-demo` before bisecting, so the test result is no longer about the history. `rm -rf regression-demo` and regenerate.
`fatal: not a git repository`	You are in the wrong directory. `cd ~/devops/lab03` and check `ls -a` shows `.git`.
Scripts fail with `bad interpreter: ^M`	Windows line endings. Run `sed -i 's/\r\$//' *.sh` and try again.

12. Looking ahead

This repository is the one you keep. Lab 04 pushes it to GitHub and puts a pull request and a code review in front of every change. Lab 06 points a build server at it, so that `make ci` runs automatically on every commit you push — at which point the commit hygiene you practised today stops being a matter of taste and starts being the thing that tells you which change broke the build.

> **Keep this folder.** Labs 04, 05, 06 and 09 all build on it. A student who deletes it after submission will be rebuilding it from scratch in week 4.